

Name : Chetan Bapurao Jadhav

Prn no : 202501040051

Division : CS-1

Batch : C13

Course : EDS

Essential of Data Science Laboratory

Practical 2:

2.1.1 List Operations:

The screenshot displays the CodeTANTRA web-based IDE interface. On the left, a panel titled "2.1.1. List operations" contains the problem description and sample test cases. The main editor area shows a Python script named "listOps.py" with the following code:

```
1 list1 = []
2 while True:
3     print("1. Add")
4     print("2. Remove")
5     print("3. Display")
6     print("4. Quit")
7
8     n = int(input("Enter choice: "))
9     if n==1:
10         add = int(input("Integer: "))
11         list1.append(add)
12         print(f"List after adding: {list1}")
13     elif n==2:
14         if len(list1)==0:
15             print("List is empty")
16         else:
17             remove = int(input("Integer: "))
18             if remove not in list1:
19                 print("Element not found")
20             else:
21                 list1.remove(remove)
22                 print(f"List after removing: {list1}")
23     elif n==3:
24         if len(list1)==0:
25             print("List is empty")
```

The interface includes a top navigation bar with the CodeTANTRA logo, a user profile, and a "Logout" button. The bottom of the IDE features a "Terminal" and "Test cases" panel, along with navigation buttons: "Prev", "Reset", "Submit", and "Next".

Average time

0.051 s

51.00 ms

Maximum time

0.071 s

71.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1

46 ms

Debug

Expected output

1. -Add
2. -Remove
3. -Display
4. -Quit
Enter-choice: - 1
Integer: - 11
List-after-adding: - [11]
1. -Add
2. -Remove
3. -Display
4. -Quit
Enter-choice: - 1
Integer: - 25
List-after-adding: - [11, -25]
1. -Add
2. -Remove
3. -Display
4. -Quit
Enter-choice: - 2

Actual output

1. -Add
2. -Remove
3. -Display
4. -Quit
Enter-choice: - 1
Integer: - 11
List-after-adding: - [11]
1. -Add
2. -Remove
3. -Display
4. -Quit
Enter-choice: - 1
Integer: - 25
List-after-adding: - [11, -25]
1. -Add
2. -Remove
3. -Display
4. -Quit
Enter-choice: - 2

2.1.2 Dictionary Operations:

CODETANTRA

Home Learn Anywhere

202501040051@mitaoe.ac.in

Support

Logout

2.1.2. Dictionary Operations

45:12

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

DictOper...

```

1 # Initial dictionary with 10 predefined records
2 student = {
3     1: "Amit",
4     2: "Riya",
5     3: "Kiran",
6     4: "Neha",
7     5: "Arjun",
8     6: "Pooja",
9     7: "Rahul",
10    8: "Sneha",
11    9: "Vikram",
12    10: "Anjali"
13 }
14 print("Original Dictionary:",student)
15 n1 = int(input())
16 student[n1] = input()
17 print("After Insertion:",student)
18
19 n2 = int(input())
20 student[n2] = input()
21 print("After Update:",student)
22
23 n3 = int(input())
24 if n3 in student:
25     del student[n3]

```

Terminal

Test cases

< Prev

Reset

Submit

Next >

Average time
0.016 s
16.00 ms

Maximum time
0.020 s
20.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 20 ms Debug

Expected output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
3
Karthik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:

Actual output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
3
Karthik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:

2.2.1 Linear Search Technique:

CDETANTRA
Home
Learn Anywhere

202501040051@mitaoe.ac.in
Support
Logout

2.2.1. Linear search Technique
14:30

Write a program to check whether the given element is present or not in the array of elements using linear search.
Input format:
• The first line of input contains the array of integers which are separated by space
• The last line of input contains the key element to be searched
Output format:
• If the element is found, print the index. If the element found multiple times, print the starting index
• If the element is not found, print **Not found**.
Sample Test Case:
Input:
1 2 3 4 3 5 6
3
Output:
2
Sample Test Cases

CTP1709...

```

1 def linear_search(arr,key):
2
3     for i in range(len(arr)):
4         if arr[i] == key:
5             return i
6     return -1
7
8 arr = list(map(int,input().split()))
9 key = int(input())
10
11 result = linear_search(arr,key)
12
13 if result != -1:
14     print(result)
15 else:
16     print("Not found")

```

Average time
0.012 s
11.75 ms

Maximum time
0.015 s
15.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 10 ms Debug

Expected output

1 2 3 4 3 5 6
3

Actual output

1 2 3 4 3 5 6
3

Terminal
Test cases

Prev
Reset
Submit
Next

2.2.2 Captain of the team:

2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

captainof...

```
1 heights = list(map(int, input().split()))
2 tallest_player = max(heights)
3 print(tallest_player)
```

Average time **0.006 s** 5.67 ms | Maximum time **0.007 s** 7.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 6 ms Debug

Expected output	Actual output
171 169 185 156 174 191 186 190 187 172 160	171 169 185 156 174 191 186 190 187 172 160
191	191

Terminal Test cases

